Reviewer Pack — Minimal Galois Group Order and Circuit Entanglement Inequali…

Entry 7d181ceca4cb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 18:27:45 UTC

Minimal Galois Group Order and Circuit Entanglement Inequality

Entry ID: `7d181ceca4cb` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-05 18:27:45 UTC

1. Conjecture

Field A (mathematical branch): Galois Theory **Field B** (complexity object): Quantum Information Theory (Circuit Entanglement)

Statement:

> For any Boolean circuit C with n inputs, the minimal order of its Galois group over the finite field GF(2) is O(n^(1/4)) and positively correlated with the entanglement entropy of C's output state.

Rationale (proposer's reasoning):

> Galois theory provides a framework for studying symmetries in mathematical structures, including circuits. The entanglement entropy quantifies the quantum correlations in a system. If a circuit exhibits a complex symmetry that is not easily described by its structure, it may have high entanglement entropy. This conjecture links these two concepts to explore their interplay in the complexity of computation.

Taxonomy category: `Galois_Quantum_Information` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `9443cbb4692da9cf`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a Boolean circuit C with n inputs, if the correlation coefficient between the minimal order of its Galois group (over GF(2)) and the entanglement entropy of C's output state is greater than or equal to 0.5 for at least 80% of the circuits tested.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"Galois Theory" AND "circuit entanglement inequality"` - `"minimal Galois group order" AND "quantum information theory"` - `"entanglement entropy" AND "Boolean circuit Galois group"`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def galois_group_order(circuit):
        n = len(circuit)
        if n == 0:
            return 1

        # Construct the field generated by the circuit's output
        field = {0, 1}
        for i in range(n):
            new_element = (field[0] ^ circuit[i]) % 2
            field.add(new_element)

        # Find the minimal order of the Galois group
        order = 1
        while True:
            if all((x ** order) % 2 == x for x in field):
                return order
            order += 1

    def entanglement_entropy(circuit):
        n = len(circuit)
        # Simplified model: assume the circuit has a uniform distribution of outputs
        return -n * math.log2(1 / (2 ** n))

    def generate_circuit(n):
        return [random.randint(0, 1) for _ in range(n)]
```

```
    correlation_values = []
    instances_tested = 0

    for n in {5, 10, 15, 20, 30, 40}:
        for _ in range(5):
            circuit = generate_circuit(n)
            group_order = galois_group_order(circuit)
            entropy = entanglement_entropy(circuit)
            correlation_values.append((group_order, entropy))
            instances_tested += 1

    if not correlation_values:
        return {
            "metric_name": "correlation_coefficient",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": 40,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    correlation_sum = sum(x * y for x, y in correlation_values)
    correlation_mean = correlation_sum / len(correlation_values)
    correlation_std = math.sqrt(sum((x * y - correlation_mean) ** 2 for x, y in correlation_values

    return {
        "metric_name": "correlation_coefficient",
        "metric_value": correlation_mean,
        "instances_tested": instances_tested,
        "n_max": 40,
        "conjecture_holds": correlation_mean >= 0.5,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...run_trial output...}}")
        results.append(trial_result)

    if all("conjecture_holds" in result and result["conjecture_holds"] for result in results):
        support_fraction = sum(1 for result in results if "conjecture_holds" in result and result[
        print(f"RESULT: SUPPORTED mean={sum(result['metric_value'] for result in results) / len(re
    elif any("conjecture_holds" in result and not result["conjecture_holds"] for result in results
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if "conjecture_hold
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE no seeds tested")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d80e9167.py", line 86, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d80e9167.py", line 53, in run_trial
    group_order = galois_group_order(circuit)
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d80e9167.py", line 29, in galois_group_
    new_element = (field[0] ^ circuit[i]) % 2
                   ~~~~~^^^
TypeError: 'set' object is not subscriptable
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed during execution, which prevented the production of data necessary to evaluate the conjecture. | next: Investigate and fix the error in the test code to allow for proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12869
2	preregistration	ollama_remote	glm4:latest	0	0	9232
3	novelty	ollama_remote	glm4:latest	0	0	8713
4	novelty	ollama_remote	glm4:latest	0	0	9019
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	32605
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10149
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9940
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10436
9	judge	ollama_remote	glm4:latest	0	0	14642

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 117605 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/7d181ceca4cb.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/7d181ceca4cb.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/7d181ceca4cb.tar.gz` (if generated)